LAB EXPERIMENT

NAME : B Hitesh Roy

REG NO : 1923272036

COURSE CODE: (CSA0618)

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHM.

Selection Sort - Question 16: Trophy Ranking System

```
def top_k_scores(scores, k):
    arr = scores.copy()
    n = len(arr)
    k = min(k, n)

    for i in range(k):
        max_index = i
        for j in range(i + 1, n):
            if arr[j] > arr[max_index]:
                max_index = j
        arr[i], arr[max_index] = arr[max_index], arr[i]

    return arr[:k]



assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
assert top_k_scores([5,3,1], 5) == [5,3,1]
assert top_k_scores([], 3) == []
```

```
print("All test cases passed!")
```

Selection Sort – Question 17: Memory-Constrained Embedded Device

```
def selection_sort_min_writes(arr):
    arr = arr.copy()
    swaps = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

    return arr, swaps



res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1
```

```
res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
assert sw2 == 0

print("All test cases passed!")
```

Selection Sort - Question 3: Library Book Reordering

```
def reorder_shelf(books):
    arr = books.copy()
    moves = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            moves += 1

    return arr, moves



ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
```

```
assert moves <= len(ordered) - 1

ordered2, moves2 = reorder_shelf([100, 200, 300])
assert moves2 == 0

print("All test cases passed!")
```

Selection Sort – Question 19: Contest Prize Distribution

```
def distribute_prizes(participants):
    arr = participants.copy()
    n = len(arr)

    for i in range(n - 1):
        max_index = i
        for j in range(i + 1, n):
            if arr[j][1] > arr[max_index][1]:
                max_index = j

        arr[i], arr[max_index] = arr[max_index], arr[i]

    return arr


ranking = distribute_prizes([
    ('Asha', 88),
    ('Ravi', 95),
```

```
    ('Meera', 79),
    ('Dev', 95)
])

scores_only = [p[1] for p in ranking]

assert scores_only == sorted(scores_only, reverse=True)
assert ranking[0][1] == 95

for name, score in ranking:
    print(name, score)

print("All test cases passed!")
```

Selection Sort - Question 20: Small Dataset in a Mobile App

```
import time

def selection_sort(arr):
    arr = arr.copy()
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
```

```
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


data = [499,129,899,45,275,60,310,150]

start = time.perf_counter()
result = selection_sort(data)
end = time.perf_counter()

print("Sorted:", result)
print("Execution Time:", end - start, "seconds")

assert selection_sort([499,129,899,45,275,60,310,150]) ==
sorted([499,129,899,45,275,60,310,150])
assert selection_sort([]) == []
assert selection_sort([7]) == [7]

print("All test cases passed!")
```

These five programs correspond to Selection Sort Questions 1–5 from your uploaded lab sheet.

2

Bubble Sort - Question 1: Exam Result Slip Correction

```
def optimized_bubble_sort(arr):
    arr = arr.copy()
```

```
    n = len(arr)
    passes = 0

    for i in range(n - 1):
        swapped = False
        passes += 1
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

    return arr, passes


sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])
assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])
assert passes < 8

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5])
assert passes_ok == 1

print("All test cases passed!")
```

Bubble Sort - Question 2: Traffic Signal Priority Queue

```
priority = {
    'ambulance': 1,
    'bus': 2,
    'car': 3
}

def bubble_sort_queue(queue):
    arr = queue.copy()
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if priority[arr[j]] > priority[arr[j + 1]]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr



queue = ['car', 'car', 'bus']
queue.append('ambulance')

result = bubble_sort_queue(queue)

assert result == ['ambulance', 'bus', 'car', 'car']
assert bubble_sort_queue(['ambulance']) == ['ambulance']
```

```
print(result)

print("All test cases passed!")
```

Bubble Sort - Question 3: Bubble Sort Visualization Tool

```
def bubble_sort_with_frames(arr):
    arr = arr.copy()
    frames = [arr.copy()]
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
        frames.append(arr.copy())

    return frames



frames = bubble_sort_with_frames([5,1,4,2,8])

assert frames[-1] == sorted([5,1,4,2,8])
assert frames[0] == [5,1,4,2,8]
assert len(frames) >= 2

for frame in frames:
    print(frame)
```

```
print("All test cases passed!")
```

Bubble Sort - Question 4: Sorting Sensor Alerts by Severity

```
def bubble_sort_plain(arr):
    arr = arr.copy()
    comparisons = 0
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - i - 1):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr, comparisons



def bubble_sort_optimized(arr):
    arr = arr.copy()
    comparisons = 0
    n = len(arr)

    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
```

```
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

    return arr, comparisons



alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]

r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)

assert r1 == r2 == sorted(alerts)
assert c2 <= c1

print("Plain comparisons:", c1)
print("Optimized comparisons:", c2)
print("All test cases passed!")
```

Bubble Sort - Question 5: Card Game Hand Sorting

```
import random

def bubble_sort_hand(hand):
```

```
    arr = hand.copy()
    n = len(arr)
    passes = 0

    for i in range(n - 1):
        swapped = False
        passes += 1
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

    return arr, passes



hand = [2,4,6,8,9,11,13]
hand.append(7)

final_hand, passes_incremental = bubble_sort_hand(hand)

assert final_hand == sorted(hand)

shuffled = hand.copy()
random.shuffle(shuffled)
```

```
_, passes_full = bubble_sort_hand(shuffled)

assert passes_incremental <= passes_full

print("Nearly Sorted Passes:", passes_incremental)
print("Random Passes:", passes_full)
print("All test cases passed!")
```

These are the Bubble Sort (Questions 1–5) programs from your uploaded lab sheet.

3

Insertion Sort - Question 1: Live Leaderboard Updates

```
def insert_updated_score(board, new_score):
    arr = board.copy()
    arr.append(new_score)

    shifts = 0
    i = len(arr) - 2

    while i >= 0 and arr[i] < new_score:   # descending order
        arr[i + 1] = arr[i]
        shifts += 1
        i -= 1

    arr[i + 1] = new_score
    return arr, shifts
```

```
board = [980, 875, 760, 690, 500]

updated_board, shifts = insert_updated_score(board, 820)
assert updated_board == [980, 875, 820, 760, 690, 500]

board2, shifts2 = insert_updated_score(board, 100)
assert board2[-1] == 100 and shifts2 == 0

print("All test cases passed!")
```

Insertion Sort - Question 2: Card Sorting While Playing

```
def pick_up_card(hand, card):
    hand = hand.copy()
    hand.append(card)

    i = len(hand) - 2

    while i >= 0 and hand[i] > card:
        hand[i + 1] = hand[i]
        i -= 1

    hand[i + 1] = card
    return hand
```

```
hand = []

for card in [7, 2, 9, 4, 1]:
    hand = pick_up_card(hand, card)

assert hand == sorted([7, 2, 9, 4, 1])
assert pick_up_card([], 5) == [5]

print("All test cases passed!")
```

Insertion Sort - Question 3: Real-Time Stock Price Feed

```
def insert_price(prices, price):
    prices = prices.copy()
    prices.append(price)

    i = len(prices) - 2

    while i >= 0 and prices[i] > price:
        prices[i + 1] = prices[i]
        i -= 1

    prices[i + 1] = price
    return prices


prices = []
```

```
for p in [102.5, 98.3, 105.1, 100.0, 97.8]:
    prices = insert_price(prices, p)

assert prices == sorted([102.5, 98.3, 105.1, 100.0, 97.8])
assert prices[0] == min(prices)
assert prices[-1] == max(prices)

print(prices)
print("All test cases passed!")
```

Insertion Sort - Question 4: Playlist Reordering by Recently Added

```
def insert_song(playlist, song):
    arr = playlist.copy()
    arr.append(song)

    i = len(arr) - 2

    while i >= 0 and arr[i][1] > song[1]:
        arr[i + 1] = arr[i]
        i -= 1

    arr[i + 1] = song
    return arr
```

```
playlist = [
    ('Intro', 120),
    ('Chill Beat', 210),
    ('Long Jam', 340)
]

updated_playlist = insert_song(playlist, ('Quick Track', 180))

durations = [song[1] for song in updated_playlist]

assert durations == sorted([120, 210, 340, 180])
assert ('Quick Track', 180) in updated_playlist

print(updated_playlist)
print("All test cases passed!")
```

Insertion Sort - Question 5: Nearly Sorted Sensor Log Correction

```
import random

def insertion_sort_count_shifts(arr):
    arr = arr.copy()
    shifts = 0

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            shifts += 1
            j -= 1

        arr[j + 1] = key

    return arr, shifts



log = [18.2, 18.5, 18.9, 17.9, 19.1, 19.4, 19.0]

sorted_log, shifts_nearly = insertion_sort_count_shifts(log)

assert sorted_log == sorted(log)

shuffled_log = log.copy()
random.shuffle(shuffled_log)

_, shifts_random = insertion_sort_count_shifts(shuffled_log)

print("Nearly-sorted shifts:", shifts_nearly)
print("Random shifts:", shifts_random)
print("All test cases passed!")
```

```
pair, distance = closest_pair(points)


print("Closest Pair:", pair)

print("Distance:", distance)
```